MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Dr. J.R. Arun kumar

Subject & Subject Code: Natural Language Processing **(6CS5-13)**

Semester: VI **Session**: 2023-24(Even Sem.)

Unit No.: V **Date of Submission**: 23/02/2024

Sl.No.	Topics Covered	Page No.	Total pages
1	**Information Retrieval and Lexical Resources:** Information Retrieval: Design features of Information Retrieval Systems,	1-3	
2	Classical, Non-classical	4-8	
3	Alternative Models of Information Retrieval	9-12	
4	valuation Lexical Resources	12-14	
5	World Net	15-18	
6	Frame Net	19-20	
7	Stemmers, POS Tagger	21-23	

References Book:

Natural Language understanding by James Allen, Pearson Education 2008

NLP: A Paninian Perspective by Akshar Bharati, Vineet Chaitanya, and Rajeev Sangal, Prentice Hal

FACULTY HOD DEAN ACADEMICS

23/3/12

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO:01

INFORMATION RETRIEVAL: DESIGN FEATURES OF INFORMATION RETRIEVAL SYSTEMS, CLASSICAL, NON-CLASSICAL, ALTERNATIVE MODELS OF INFORMATION RETRIEVAL, VALUATION LEXICAL RESOURCES: WORLD NET, FRAME NET, STEMMERS, POS TAGGER.

What is Information Retrieval?

Information Retrieval (IR) can be defined as a software program that deals with the organization, storage, retrieval, and evaluation of information from document repositories, particularly textual information. Information Retrieval is the activity of obtaining material that can usually be documented on an unstructured nature **i.e.** usually text which satisfies an information need from within large collections which is stored on computers. For example, Information Retrieval can be when a user enters a query into the system.

Not only librarians, professional searchers, etc engage themselves in the activity of information retrieval but nowadays hundreds of millions of people engage in IR every day when they use web search engines. Information Retrieval is believed to be the dominant form of Information access. The **IR system** assists the users in finding the information they require but it does not explicitly return the answers to the question. It notifies regarding the existence and location of documents that might consist of the required information. Information retrieval also extends support to users in browsing or filtering document collection or processing a set of retrieved documents. The system searches over billions of documents stored on millions of computers. A spam filter, manual or automatic means are provided by Email program for classifying the mails so that it can be placed directly into particular folders.

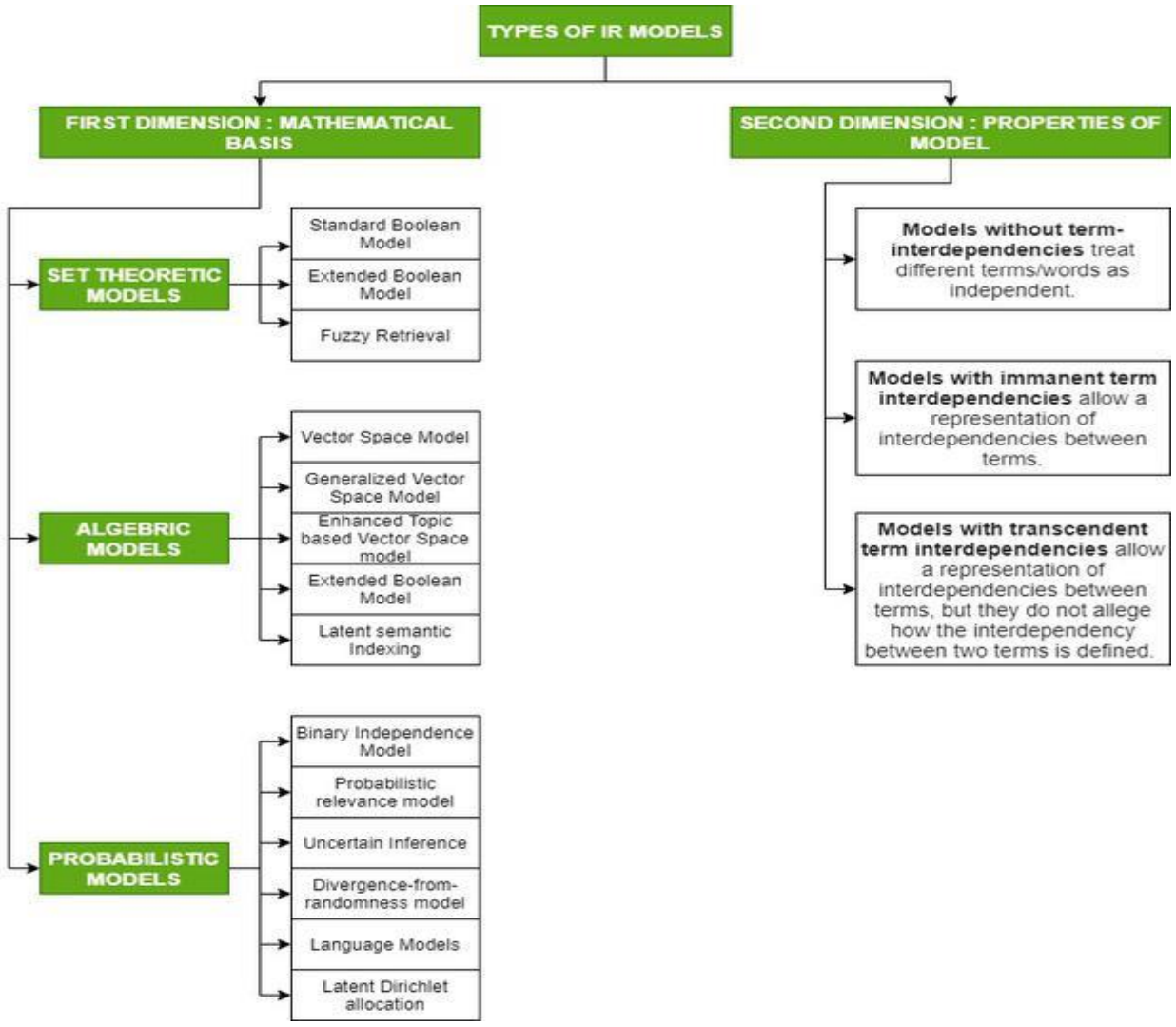
An IR system has the ability to represent, store, organize, and access information items. A set of **keywords** are required to search. Keywords are what people are searching for in search engines. These keywords summarize the description of the information.

What is an IR Model?

An Information Retrieval (IR) model selects and ranks the document that is required by the user or the user has asked for in the form of a query. The documents and the queries are represented in a similar manner, so that document selection and ranking can be formalized by a matching function that returns a **retrieval status value (RSV)** for each document in the collection. Many of the Information Retrieval systems represent document contents by a set of descriptors, called terms, belonging to a vocabulary V. An IR model determines the query-document matching function according to four main approaches:

The estimation of the probability of user's relevance rel for each document ***d*** *and query* ***q*** *with respect to a set R* ***q*** *of training documents:* ***Prob (rel|d, q, Rq)***

Types of IR Models

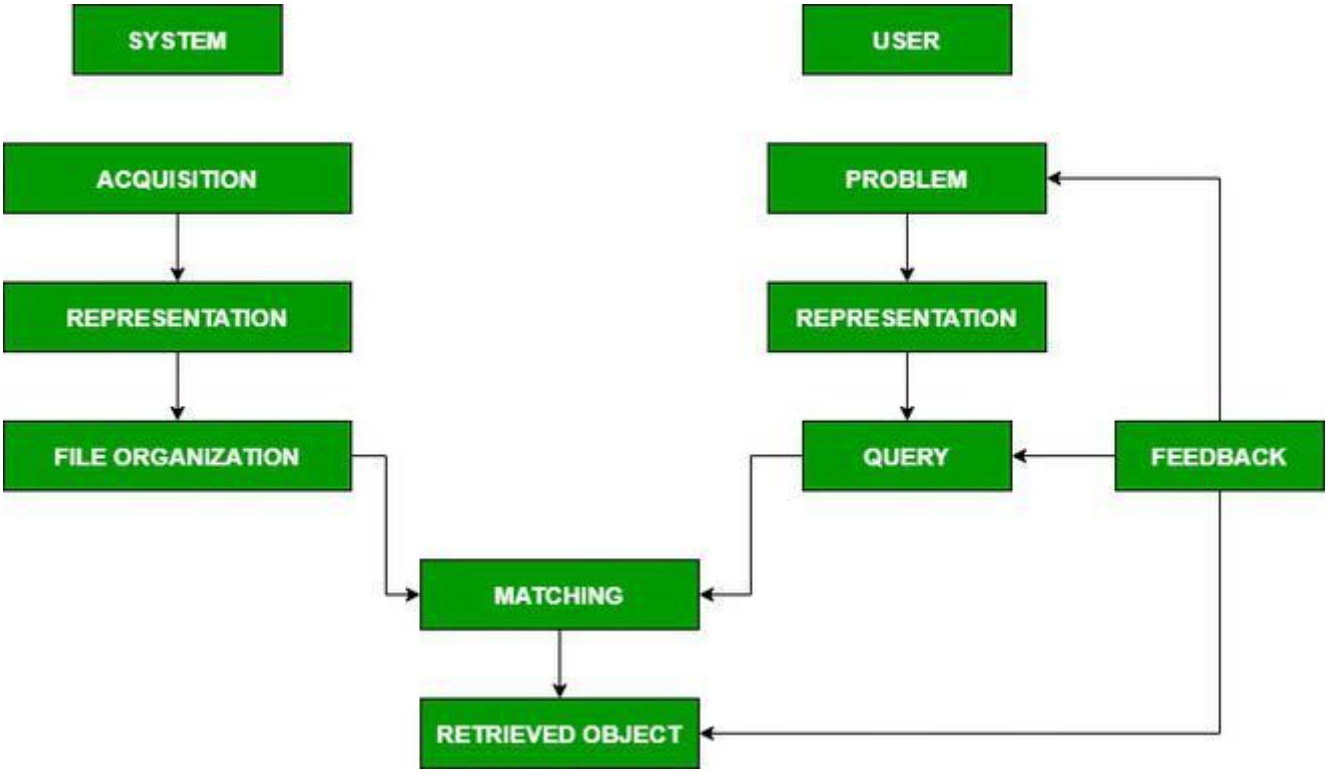


UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO:02

Components of Information Retrieval/ IR Model



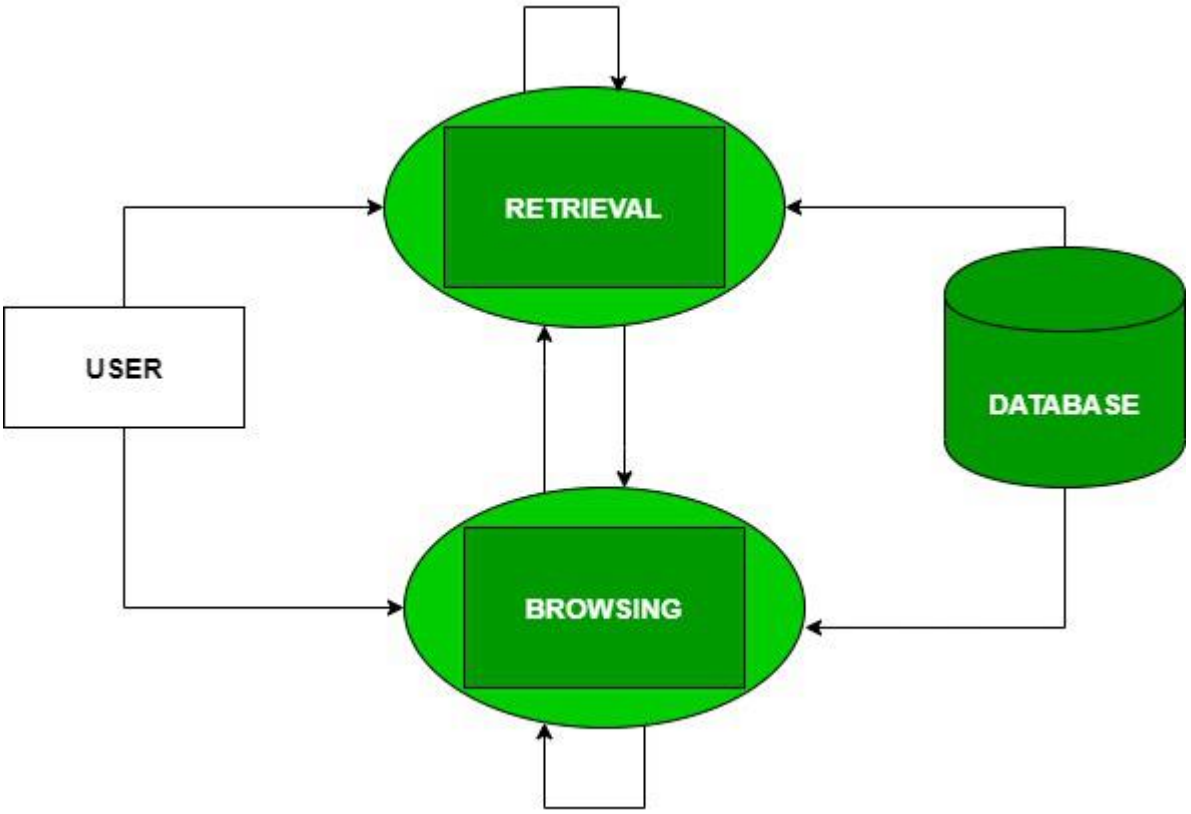
- **Acquisition:** In this step, the selection of documents and other objects from various web resources that consist of text-based documents takes place. The required data is collected by web crawlers and stored in the database.
- **Representation:** It consists of indexing that contains free-text terms, controlled vocabulary, manual & automatic techniques as well. example: Abstracting contains summarizing and Bibliographic description that contains author, title, sources, data, and metadata.
- **File Organization:** There are two types of file organization methods. i.e. *Sequential*: It contains documents by document data. *Inverted*: It contains term by term, list of records under each term. *Combination* of both.
- **Query:** An IR process starts when a user enters a query into the system. Queries are formal statements of information needs, for example, search strings in web search engines. In information

retrieval, a query does not uniquely identify a single object in the collection. Instead, several objects may match the query, perhaps with different degrees of relevancy.

Difference Between Information Retrieval and Data Retrieval

Information Retrieval	**Data Retrieval**
The software program that deals with the organization, storage, retrieval, and evaluation of information from document repositories particularly textual information.	Data retrieval deals with obtaining data from a database management system such as ODBMS. It is A process of identifying and retrieving the data from the database, based on the query provided by user or application.
Retrieves information about a subject.	Determines the keywords in the user query and retrieves the data.
Small errors are likely to go unnoticed.	A single error object means total failure.
Not always well structured and is semantically ambiguous.	Has a well-defined structure and semantics.
Does not provide a solution to the user of the database system.	Provides solutions to the user of the database system.
The results obtained are approximate matches.	The results obtained are exact matches.
Results are ordered by relevance.	Results are unordered by relevance.
It is a probabilistic model.	It is a deterministic model.

User Interaction With Information Retrieval System



The User Task: The information first is supposed to be translated into a query by the user. In the information retrieval system, there is a set of words that convey the semantics of the information that is required whereas, in a data retrieval system, a query expression is used to convey the constraints which are satisfied by the objects. Example: A user wants to search for something but ends up searching with another thing. This means that the user is browsing and not searching. The above figure shows the interaction of the user through different tasks.

- **Logical View of the Documents:** A long time ago, documents were represented through a set of index terms or keywords. Nowadays, modern computers represent documents by a full set of words which reduces the set of representative keywords. This can be done by eliminating stopwords i.e. articles and connectives. These operations are text operations. These text operations reduce the complexity of the document representation from full text to set of index terms.

Past, Present, and Future of Information Retrieval

1. Early Developments: As there was an increase in the need for a lot of information, it became necessary to build data structures to get faster access. The index is the data structure for faster retrieval of information. Over centuries manual categorization of hierarchies was done for indexes.

2. Information Retrieval In Libraries: Libraries were the first to adopt IR systems for information retrieval. In first-generation, it consisted, automation of previous technologies, and the search was based on author name and title. In the second generation, it included searching by subject heading, keywords, etc. In the third generation, it consisted of graphical interfaces, electronic forms, hypertext features, etc.

3. The Web and Digital Libraries: It is cheaper than various sources of information, it provides greater access to networks due to digital communication and it gives free access to publish on a larger medium.

Advantages of Information Retrieval

1. Efficient Access: Information retrieval techniques make it possible for users to easily locate and retrieve vast amounts of data or information.

2. Personalization of Results: User profiling and personalization techniques are used in information retrieval models to tailor search results to individual preferences and behaviors.

3. Scalability: Information retrieval models are capable of handling increasing data volumes.

4. Precision: These systems can provide highly accurate and relevant search results, reducing the likelihood of irrelevant information appearing in search results.

Disadvantages of Information Retrieval

1. Information Overload: When a lot of information is available, users often face information overload, making it difficult to find the most useful and relevant material.

2. Lack of Context: Information retrieval systems may fail to understand the context of a user's query, potentially leading to inaccurate results.

3. Privacy and Security Concerns: As information retrieval systems often access sensitive user data, they can raise privacy and security concerns.

4. Maintenance Challenges: Keeping these systems up-to-date and effective requires ongoing efforts, including regular updates, data cleaning, and algorithm adjustments.

5. Bias and fairness: Ensuring that information retrieval systems do not exhibit biases and provide fair and unbiased results is a crucial challenge, especially in contexts like web search engines and recommendation systems.

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO:03

DESIGN FEATURES OF INFORMATION RETRIEVAL (IR) SYSTEMS

Let us look at some design features of information retrieval systems.

Inverted Index

An inverted index is an **index data structure** storing a mapping from content (content can be words or numbers) to its locations in a document or a set of documents.

We can also say the inverted index is a **hashmap-like data structure** that directs the user from a word to a document or a web page. The inverted index is also the **primary** data structure of most information retrieval systems.

Inverted index as a data structure lists for **every word, all documents** that contain it, and **frequency** of the occurrences in the document hence making it easy to search for hits of a query word.

Types of inverted indexes: Mainly two types record level inverted index, and word level inverted index.

Record level inverted index contains a list of references to **documents for each word**.

Word level inverted index additionally contains the **positions of each word within a document**. This form of the inverted index also offers more functionality but needs more processing power and space to be created.

ADVANTAGES OF INVERTED INDEX: The main utility of inverted index is that it allows fast full-text searches at the cost of increased processing when a document is added to the database.

Inverted index is easy to develop.

Inverted index is also the most popular data structure used in document retrieval systems used on a large scale for example, in search engines.

DISADVANTAGE OF INVERTED INDEX: Inverted index has a large storage overhead with **high maintenance costs** for the update, delete, and insert statements.

STOP WORD ELIMINATION

Stop words are high-frequency words that are **deemed unlikely to be useful** for searching inside the documents of the information retrieval system.

All the words in the corpus with **less semantic weights** are kept in a list called a stop list.

Example: Articles like a, an, the, and prepositions like in, of, for, at, etc. are examples of stop words.

Size reduction of the inverted index using stop list: One main pro of eliminating stop words is that the size of the inverted index can be significantly reduced by a stop list.

As per Zipf's law, a stop list covering a few dozen words reduces the size of the inverted index by **almost half**.

One disadvantage of stop word elimination is that sometimes it may **cause the elimination of the term that is useful** for searching.

Example: If we eliminate the alphabet A from Vitamin A, then the word will lose its significance.

STEMMING

Stemming is the heuristic process of **extracting the base form of words** by chopping off the ends of words. It is the process of producing **morphological variants** of a root or base word. Stemming programs are commonly referred to as stemming algorithms or stemmers. Stemming is one of the important steps in information retrieval systems like search engines.

For example, the words laughing, laughs, and laughed would be stemmed from the root word laugh.

Usage of stemming in information retrieval system with an example: If we want to search for the word *chocolate* in a collection of documents, we want to see all the documents that have information about the word chocolate.

It may so happen that the words chocolates, chocolatey, and choco may be present in many documents instead of chocolate.

To relate these many words, we can stem these words into their root word *chocolate* again so that we can retrieve all the documents containing this base word no matter the way it is represented across documents.

There are **many standard tools** for performing this reduction (of stemming into root word) like the Porter's Stemmer, the Snowball stemmer, the Lancaster stemmer, etc.

CLASSICAL, NON-CLASSICAL

INFORMATION RETRIEVAL (IR) MODEL

Mathematically, models are used in many scientific areas having objective to understand some phenomenon in the real world. A model of information retrieval predicts and explains what a user will

find in relevance to the given query. IR model is basically a pattern that defines the above-mentioned aspects of retrieval procedure and consists of the following −

A model for documents.

A model for queries.

A matching function that compares queries to documents.

Mathematically, a retrieval model consists of −

D − Representation for documents.

R − Representation for queries.

F − The modeling framework for D, Q along with relationship between them.

R (q,di) − A similarity function which orders the documents with respect to the query. It is also called ranking.

TYPES OF INFORMATION RETRIEVAL (IR) MODEL

An information model (IR) model can be classified into the following three models −

CLASSICAL IR MODEL

It is the simplest and easy to implement IR model. This model is based on mathematical knowledge that was easily recognized and understood as well. Boolean, Vector and Probabilistic are the three classical IR models.

NON-CLASSICAL IR MODEL

It is completely opposite to classical IR model. Such kind of IR models are based on principles other than similarity, probability, Boolean operations. Information logic model, situation theory model and interaction models are the examples of non-classical IR model.

ALTERNATIVE IR MODEL

It is the enhancement of classical IR model making use of some specific techniques from some other fields. Cluster model, fuzzy model and latent semantic indexing (LSI) models are the example of alternative IR model.

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO:04

ALTERNATIVE INFORMATION RETRIEVAL MODEL

The alternative Information Retrieval Model is the enhancement of the classical IR model that makes use of some specific techniques from some other fields.

- Examples of Alternative IR models: Cluster models, Fuzzy models, and Latent Semantic Indexing (LSI) models.

THE BOOLEAN MODEL

The Boolean model in information retrieval is **based on the set theory and boolean algebra**. We can pose any query in the form of a Boolean expression of terms where the terms are logically combined using the **Boolean operators AND, OR, and NOT** in the Boolean retrieval model.

- Using the Boolean operators, the **terms** in the query and the concerned documents can be **combined** to form a whole new set of documents.
    - The Boolean **AND** of two logical statements x and y means that **both x AND y must be satisfied** and will be a set of documents that will **smaller or equal** to the document set
    - while the Boolean **OR** of these same two statements means that **at least one of these statements** must be satisfied and will fetch a set of documents that will be **greater or equal** to the document set otherwise.
    - **Any number** of logical statements can be **combined** using the three Boolean operators.
- The queries are designed as boolean expressions which have precise semantics and the retrieval strategy is based on **binary decision criterion**.
- The Boolean model can also be explained well by **mapping the terms in the query with a set of documents**.

The most famous web search engine in recent times Google also ranks the web page result set based on a two-stage system: In the **first** step, a **Simple Boolean Retrieval** model** returns matching documents** in no particular order, and in the next step **ranking** is done according to some **estimator of relevance**.

ASPECTS OF BOOLEAN INFORMATION RETRIEVAL MODEL

Indexing: Indexing is one of the **core functionalities** of the information retrieval models and the **first step** in building an IR system assisting with the efficient retrieval of information.

- Indexing is majorly an **offline operation** that collects data about **which words occur in the text corpus** so that at search time we only have to access the **pre-compiled index** done beforehand.
- The boolean model builds the indices for the terms in the query considering that **index terms are present or absent** in a document.

Term-Document Incidence matrix: This is one of the basic **mathematical models to represent text data** and can be used to answer Boolean expression queries using the Boolean Retrieval Model. It can be **used to answer any query** as a Boolean expression.

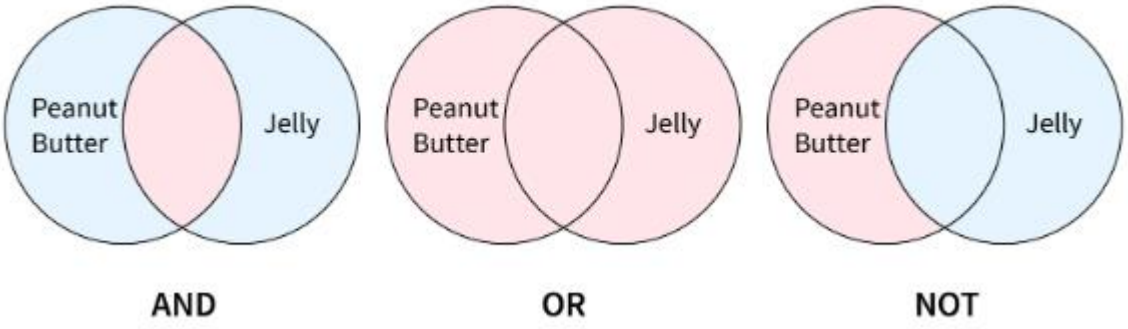
- It views the document as the **set of terms and creates the indexing** required for the Boolean retrieval model.
- The text data is represented in the form of a **matrix** where **rows** of the matrix represent the sentences and the **columns** of the matrix represent the word for the data which needs to be analyzed and retrieved and the **values** of the matrix represent the number of occurrences of the words.
- This model has **good precision** as the documents are retrieved if the condition is matched but, it **doesn't scale well** with the size of the corpus, and an inverted index can be used as a good alternative method.

Processing the data for Boolean retrieval model

- We should **strip** unwanted characters/markup like HTML tags, punctuation marks, numbers, etc. before breaking the corpus into tokens/keywords on whitespace.
- **Stemming** needs to be done and then common stopwords are to be removed depending on the application need
- The term document incidence matrix or **inverted index** (with the keyword a list of docs containing it) is built.
- Then the common queries/phrases may be detected using a **domain-specific dictionary** if needed.

Example of Information Retrieval in Boolean Model

- For example, the term **Peanut Butter individually** (or **Jelly** individually) defines all the documents with the term Peanut Butter (or Jelly) alone and indexes them.
- If the information needed is based on **Peanut Butter AND Jelly**, we will be giving a set of documents that contain **both** the words and so the query with the keywords Peanut Butter AND Jelly will be giving a set of **documents** that are having the **both the words** Peanut Butter AND Jelly.
- Using OR the search will return documents containing **either Peanut Butter or documents containing Jelly** or documents containing both Peanut Butter and Jelly.



Advantages of Boolean Model

- It is **easy** to implement and it is computationally **efficient**. Hence, it is the **standard** model for the current large-scale, operational retrieval systems and many of the major online information services use it.
- It enables users to **express structural and conceptual constraints** to describe important linguistic features. Users find that synonym specifications (reflected by OR-clauses) and phrases (represented by proximity relations) are useful in the formulation of queries
- The Boolean approach possesses a **great expressive power** and clarity. Boolean retrieval is **very effective** if a query requires an exhaustive and unambiguous selection.
- The Boolean method offers a **multitude of techniques to broaden or narrow down a query**.
- The Boolean approach can be especially **effective** in the **later stages of the search process**, because of the clarity and exactness with which relationships between concepts can be represented.

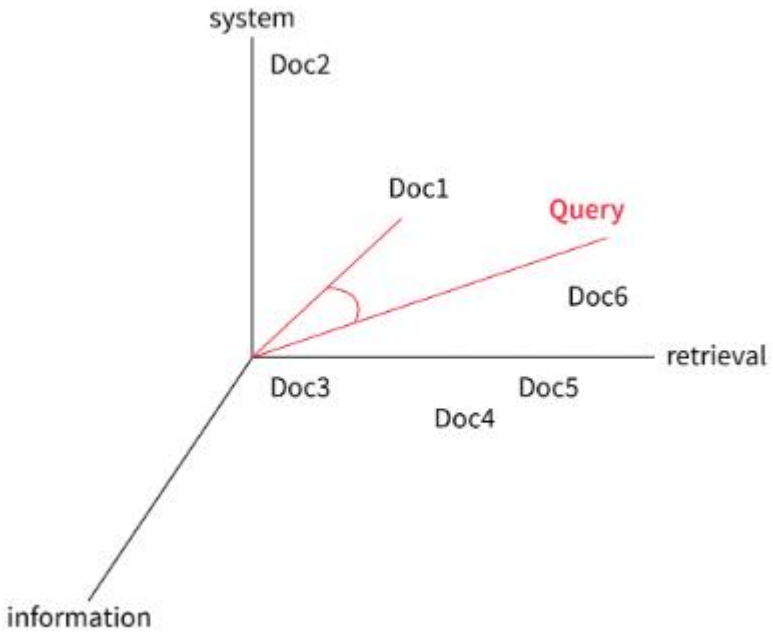
Shortcomings of Standard Boolean Approach

- Users find it **difficult to construct effective Boolean queries** for several reasons. Users are using the natural language terms AND, OR, or NOT that have different meanings when used in a query.
- Hence the users will **make errors** when they form a Boolean query because they resort to their own knowledge of English.

THE VECTOR SPACE MODEL

Also called term vector models, the vector space model is an **algebraic model for representing text document**s (or also many kinds of multimedia objects in general) as vectors of identifiers such as index terms.

The vector space model is based on the **notion of similarity between the search document** and the representative query prepared by the user which should be similar to the documents needed for information retrieval.



SCALER

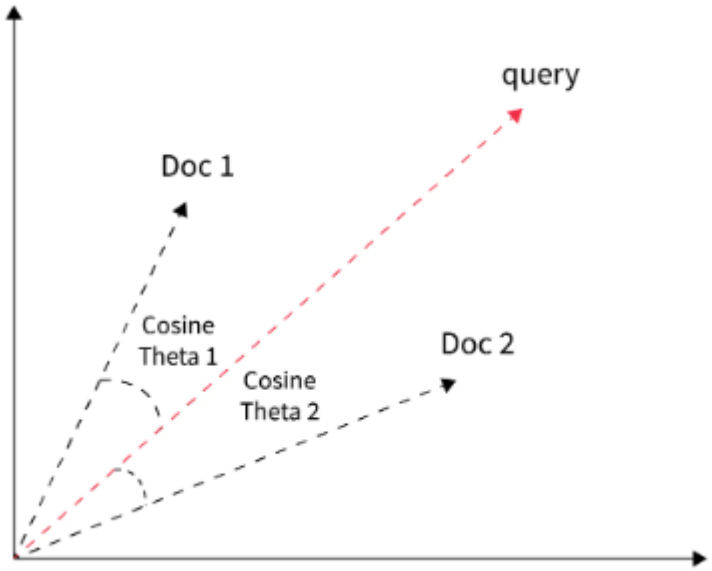
We can represent both documents and queries with vectors with a t-dimensional vector representation:

- dj = (w1,j,w2,j,...,wt,j) for a document with
- q = (w1,q,w2,q,...,wt,q)
- **Definition of Terms**: The **components of each dimension** in the document and query representation correspond to a separate term whose **terms can be** single words, keywords, or longer phrases also.
    - If the **words** of the documents are chosen to be the terms in the above representation, the **dimensionality** of the vectors is the number of **distinct words** occurring in the vocabulary of the corpus.
- **Computing the values of the Terms**: The value of the vector is **non-zero** if a term or word occurs in the document. Many **different schemes can be taken** for example count of occurrences, normalized counts, etc. but the most **popular** and efficient representation is **tf-idf term weighing**.
- The vector space model represents the documents and queries as vectors in a **multidimensional space whose dimensions are the terms** further used to build an index to represent the documents.

Notion of Similarity in Vector Space Model

The assumptions of the **document similarities theory** are used to compute the relevancy rankings of documents and the keywords in the search in vector space models.

- **Angle of deviation between query and document:** One way is to compare the **deviation of angles between each document vector and the original query vector** where the query is represented as some kind of vector as the documents.
- **Cosine distance as similarity metric:** The most popular and easier method in practice is to **calculate the cosine of the angle between the vectors** - A cosine value of zero means that the query and document vector are orthogonal and have no match at all.
    - A **zero cosine similarity** value implies that the **terms** in the query **do not exist** in the document we are considering.
- **Ranking the results using a similarity metric:** The degree of **similarity** between the representation of the prepared **document** and the representations of the documents in the **collection** is used to rank the search results.



One other way of looking at the similarity criterion in the vector space model is that - the more the two representations of search documents and the user-prepared query **agree** in the **given elements and their distribution**, the **higher** would the **probability of their representing** similar information.

Index Creation for Terms in the Vector Space Model

The **creation** of the indices for the vector space model **involves** lexical scanning, morphological analysis, and term value computation.

- **Lexical scanning** is the creation of individual word documents to identify the **significant terms and morphological analysis reduces** to reduce different word forms to common stems and then compute the values of terms on the basis of stemmed words.

- The terms of the query are also **weighted** to take into account their **importance**, and they are computed by using the **statistical distributions of the terms** in the collection and in the documents.
- The vector space model assigns a **high ranking score** to a document that contains only a few of the query terms if these terms occur **infrequently** in the collection of the original corpus but frequently in the document.

Assumptions of the Vector Space Model

- The **more similar** a document vector is to a query vector, the more likely it is that the document is **relevant** to that query.
- The **words** used to define the dimensions of the space are **orthogonal or independent**.
- The **similarity** assumption is an approximation and **realistic** whereas the assumption that words are pairwise **independent doesn't hold true** in realistic scenarios.

Disadvantages of Vector Space Model

- Long documents are **poorly represented** because they have poor similarity values due to a small scalar product and a **large dimensionality of the terms** in the model.
- Search keywords must be **precisely designed** to match document terms and the word substrings might result in a **false positive match**.
- **Semantic sensitivity:** Documents with similar context but different term vocabulary won't be associated resulting in **false negative matches**.
- The **order in which the terms appear** in the document is lost in the vector space representation.
- **Weighting** is intuitive but not represented **formally** in the model.
- **Issues with implementation:** Due to the need for the similarity metric calculation and in turn storage of all the values of all vector components, it is **problematic** in case of **incremental updates** of the index
    - Adding a **single new document** changes the document frequencies of terms that occur in the document, which **changes the vector lengths of every document** that contains one or more of these terms.

PROBABILISTIC MODEL

Probabilistic models provide the **foundation for reasoning under uncertainty** in the realm of information retrieval.

Let us understand why there is **uncertainty** while retrieving documents and the basis for probability models in information retrieval.

Uncertainty in retrieval models: The probabilistic models in information retrieval are built on the idea that the process of retrieval is inherently uncertain from multiple standpoints:

- There is uncertainty in the **understanding of user's information needs** - We can not sure that the user mapped their needs into the query they have presented.
- Even if the query represents the need well, there is **uncertainty in the estimation of document relevance** for the query which stems from either the uncertainty from the selection of the document representation or the **uncertainty from matching the query and documents**.

Basis of probabilistic retrieval model: Probabilistic model is based on the **Probability Ranking Principle** which states that an information retrieval system is supposed to rank the documents based on their **probability of relevance to the query given** all the other pieces of evidence available.

- Probabilistic information retrieval models **estimate how likely it is that a document** is relevant for a query.
- There may be a variety of sources of evidence that are used by the probabilistic retrieval methods and the most common one is the **statistical distribution of the terms in both** the relevant and non-relevant documents.
- Probabilistic information models are also among the **oldest and best performing** and most widely used IR models.

Types of Probabilistic information retrieval models: The classic probabilistic models (BIM, Two Poisson, BM11, BM25), The Language models for information retrieval, and the Bayesian networks-based models for information retrieval.

Language Models for Information Retrieval

Language modeling is a **formal probabilistic retrieval framework** based on concepts in speech recognition and natural language processing. However, language models have been used for **speech recognition** and natural language processing tasks (machine translation) for more than **thirty years**. They have been used in the IR domain since the 2000s with great success in **recent times**.

- The language models for information retrieval are based on the **idea** that a query is **considered generated from an ideal document that satisfies the information need** and the IR system's job is then to estimate the **likelihood** of each document in the **collection being the ideal document** and rank the documents in ascending or decreasing order.
    - Each document is viewed as a **language sample**, each query as a **generation process**. The retrieved documents are ranked based on the probabilities of producing a query from the corresponding language models of these documents.
    - Many research studies have also confirmed that **language modeling techniques are preferred over tf-idf** (term frequency-inverse document frequency) weights because of empirical

performance as well as the probabilistic meaning that can be formally derived from a language modeling framework.

The majority of language modeling approaches to information retrieval can be categorized into one of four groups:

- The **generative query likelihood approach**, which ranks based on the likelihood of a document language model generating the query
- The **generative document likelihood approach**, which ranks based on the likelihood of a query language model generating a document
- The **comparative approach**, which ranks based on the similarity between the query language model and document language model
- The **translation models**, which rank based on the likelihood of the query being viewed as a translation of a document
- The cluster-based language models.

CONCLUSION

- `Information Retrieval systems are software systems that provide access to books, journals, many kinds of documents, and multimedia objects. Web search engines are the most visible IR applications.
- Information Retrieval models provide the framework for collecting, storing, managing, matching, and ranking the documents and building many aspects of IR models.
- There are mainly three classes of IR models - Boolean, Vector space, and Probabilistic.
- Documents and queries are represented as sets of index terms based on set theory rules in the Boolean retrieval model.
- In the vector model, documents and queries are represented as vectors in a t-dimensional space and modeled on algebraic rules.
- The framework for modeling document and query representations is based on probability theory in the probabilistic information retrieval models.

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO: 5

NLP | WordNet for tagging

- WordNet is the lexical database i.e. dictionary for the English language, specifically designed for natural language processing.

WordNet tag	Treebank tag
n	NN
a	JJ
s	JJ
r	RB
v	VB

Code #1 : Creating class to look up words in WordNet.

```
from nltk.tag import SequentialBackoffTagger

from nltk.corpus import wordnet

from nltk.probability import FreqDist



class WordNetTagger(SequentialBackoffTagger):
```

```
'''
>>> wt = WordNetTagger()
>>> wt.tag(['food', 'is', 'great'])
[('food', 'NN'), ('is', 'VB'), ('great', 'JJ')]
'''

def __init__(self, *args, **kwargs):

    SequentialBackoffTagger.__init__(self, *args, **kwargs)
    self.wordnet_tag_map = {
    'n': 'NN',
    's': 'JJ',
    'a': 'JJ',
    'r': 'RB',
    'v': 'VB'
    }

def choose_tag(self, tokens, index, history):

word = tokens[index]
```

```
    fd = FreqDist()



    for synset in wordnet.synsets(word):

        fd[synset.pos()] += 1




    return self.wordnet_tag_map.get(fd.max())
```

This WordNetTagger class will count the no. of each POS tag found in the Synsets for a word and then, the most common tag is to treebank tag using internal mapping.

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO: 06

Sample usage for framenet

FRAMENET

The FrameNet corpus is a lexical database of English that is both human- and machine-readable, based on annotating examples of how words are used in actual texts. FrameNet is based on a theory of meaning called Frame Semantics, deriving from the work of Charles J. Fillmore and colleagues. The basic idea is straightforward: that the meanings of most words can best be understood on the basis of a semantic frame: a description of a type of event, relation, or entity and the participants in it. For example, the concept of cooking typically involves a person doing the cooking (Cook), the food that is to be cooked (Food), something to hold the food while cooking (Container) and a source of heat (Heating_instrument). In the FrameNet project, this is represented as a frame called Apply_heat, and the Cook, Food, Heating_instrument and Container are called frame elements (FEs). Words that evoke this frame, such as fry, bake, boil, and broil, are called lexical units (LUs) of the Apply_heat frame. The job of FrameNet is to define the frames and to annotate sentences to show how the FEs fit syntactically around the word that evokes the frame.

Frames

A Frame is a script-like conceptual structure that describes a particular type of situation, object, or event along with the participants and props that are needed for that Frame. For example, the "Apply_heat" frame describes a common situation involving a Cook, some Food, and a Heating_Instrument, and is evoked by words such as bake, blanch, boil, broil, brown, simmer, steam, etc.

We call the roles of a Frame "frame elements" (FEs) and the frame-evoking words are called "lexical units" (LUs).

FrameNet includes relations between Frames. Several types of relations are defined, of which the most important are:

- Inheritance: An IS-A relation. The child frame is a subtype of the parent frame, and each FE in the parent is bound to a corresponding FE in the child. An example is the "Revenge" frame which inherits from the "Rewards_and_punishments" frame.
- Using: The child frame presupposes the parent frame as background, e.g the "Speed" frame "uses" (or presupposes) the "Motion" frame; however, not all parent FEs need to be bound to child FEs.
- Subframe: The child frame is a subevent of a complex event represented by the parent, e.g. the "Criminal_process" frame has subframes of "Arrest", "Arraignment", "Trial", and "Sentencing".
- Perspective_on: The child frame provides a particular perspective on an un-perspectivized parent frame. A pair of examples consists of the "Hiring" and "Get_a_job" frames, which perspectivize the "Employment_start" frame from the Employer's and the Employee's point of view, respectively.

To get a list of all of the Frames in FrameNet, you can use the *frames()* function. If you supply a regular expression pattern to the *frames()* function, you will get a list of all Frames whose names match that pattern:

```
>>> from pprint import pprint
>>> from operator import itemgetter
>>> from nltk.corpus import framenet as fn
>>> from nltk.corpus.reader.framenet import PrettyList
>>> x = fn.frames(r'(?i)crim')
>>> x.sort(key=itemgetter('ID'))
>>> x
[<frame ID=200 name=Criminal_process>, <frame ID=500 name=Criminal_investigation>,
...]
>>> PrettyList(sorted(x, key=itemgetter('ID')))
[<frame ID=200 name=Criminal_process>, <frame ID=500 name=Criminal_investigation>,
...]
```

To get the details of a particular Frame, you can use the *frame()* function passing in the frame number:

```
>>> from pprint import pprint
>>> from nltk.corpus import framenet as fn
>>> f = fn.frame(202)
>>> f.ID
202
>>> f.name
'Arrest'
>>> f.definition
"Authorities charge a Suspect, who is under suspicion of having committed a
crime..."
>>> len(f.lexUnit)
11
>>> pprint(sorted([x for x in f.FE]))
['Authorities',
 'Charges',
 'Co-participant',
 'Manner',
 'Means',
 'Offense',
 'Place',
 'Purpose',
 'Source_of_legal_authority',
 'Suspect',
 'Time',
 'Type']
>>> pprint(f.frameRelations)
[<Parent=Intentionally_affect -- Inheritance -> Child=Arrest>,
<Complex=Criminal_process -- Subframe -> Component=Arrest>, ...]
```

The *frame()* function shown above returns a dict object containing detailed information about the Frame. See the documentation on the *frame()* function for the specifics.

You can also search for Frames by their Lexical Units (LUs). The *frames_by_lemma()* function returns a list of all frames that contain LUs in which the 'name' attribute of the LU matches the given regular expression. Note that LU names are composed of "lemma.POS", where the "lemma" part can be made up of either a single lexeme (e.g. 'run') or multiple lexemes (e.g. 'a little') (see below).

```
>>> PrettyList(sorted(fn.frames_by_lemma(r'(?i)a little'), key=itemgetter('ID')))
[<frame ID=189 name=Quanti...>, <frame ID=2001 name=Degree>]
```

Lexical Units

A lexical unit (LU) is a pairing of a word with a meaning. For example, the "Apply_heat" Frame describes a common situation involving a Cook, some Food, and a Heating Instrument, and is _evoked_ by words such as bake, blanch, boil, broil, brown, simmer, steam, etc. These frame-evoking words are the LUs in the Apply_heat frame. Each sense of a polysemous word is a different LU.

We have used the word "word" in talking about LUs. The reality is actually rather complex. When we say that the word "bake" is polysemous, we mean that the lemma "bake.v" (which has the word-forms "bake", "bakes", "baked", and "baking") is linked to three different frames:

- Apply_heat: "Michelle baked the potatoes for 45 minutes."
- Cooking_creation: "Michelle baked her mother a cake for her birthday."
- Absorb_heat: "The potatoes have to bake for more than 30 minutes."

These constitute three different LUs, with different definitions.

Multiword expressions such as "given name" and hyphenated words like "shut-eye" can also be LUs. Idiomatic phrases such as "middle of nowhere" and "give the slip (to)" are also defined as LUs in the appropriate frames ("Isolated_places" and "Evading", respectively), and their internal structure is not analyzed.

Framenet provides multiple annotated examples of each sense of a word (i.e. each LU). Moreover, the set of examples (approximately 20 per LU) illustrates all of the combinatorial possibilities of the lexical unit.

Each LU is linked to a Frame, and hence to the other words which evoke that Frame. This makes the FrameNet database similar to a thesaurus, grouping together semantically similar words.

In the simplest case, frame-evoking words are verbs such as "fried" in:

"Matilde fried the catfish in a heavy iron skillet."

Sometimes event nouns may evoke a Frame. For example, "reduction" evokes "Cause_change_of_scalar_position" in:

"…the reduction of debt levels to $665 million from $2.6 billion."

Adjectives may also evoke a Frame. For example, "asleep" may evoke the "Sleep" frame as in:

"They were asleep for hours."

Many common nouns, such as artifacts like "hat" or "tower", typically serve as dependents rather than clearly evoking their own frames.

Details for a specific lexical unit can be obtained using this class's *lus()* function, which takes an optional regular expression pattern that will be matched against the name of the lexical unit:

```
>>> from pprint import pprint
>>> PrettyList(sorted(fn.lus(r'(?i)a little'), key=itemgetter('ID')))
[<lu ID=14733 name=a little.n>, <lu ID=14743 name=a little.adv>, ...]
```

You can obtain detailed information on a particular LU by calling the *lu()* function and passing in an LU's 'ID' number:

```
>>> from pprint import pprint
>>> from nltk.corpus import framenet as fn
>>> fn.lu(256).name
'foresee.v'
```

```
>>> fn.lu(256).definition
'COD: be aware of beforehand; predict.'
>>> fn.lu(256).frame.name
'Expectation'
>>> fn.lu(256).lexemes[0].name
'foresee'
```

Note that LU names take the form of a dotted string (e.g. “run.v” or “a little.adv”) in which a lemma precedes the “.” and a part of speech (POS) follows the dot. The lemma may be composed of a single lexeme (e.g. “run”) or of multiple lexemes (e.g. “a little”). The list of POSs used in the LUs is:

v - verb n - noun a - adjective adv - adverb prep - preposition num - numbers intj - interjection art - article c - conjunction scon - subordinating conjunction

For more detailed information about the info that is contained in the dict that is returned by the *lu()* function, see the documentation on the *lu()* function.

Annotated Documents

The FrameNet corpus contains a small set of annotated documents. A list of these documents can be obtained by calling the *docs()* function:

```
>>> from pprint import pprint
>>> from nltk.corpus import framenet as fn
>>> d = fn.docs('BellRinging')[0]
>>> d.corpname
'PropBank'
>>> d.sentence[49]
full-text sentence (...) in BellRinging:


[POS] 17 tags

[POS tagset] PENN

[text] + [annotationSet]

`` I live in hopes that the ringers themselves will be drawn into
            *****          *******                     *****
            Desir          Cause_t                     Cause
            [1]            [3]                         [2]

 that fuller life .
      ******
      Comple
      [4]
 (Desir=Desiring, Cause_t=Cause_to_make_noise, Cause=Cause_motion,
Comple=Completeness)
>>> d.sentence[49].annotationSet[1]
annotation set (...):

[status] MANUAL

[LU] (6605) hope.n in Desiring

[frame] (366) Desiring

[GF] 2 relations
```

```
[PT] 2 phrases

[text] + [Target] + [FE] + [Noun]

`` I live in hopes that the ringers themselves will be drawn into
   _ ^^^^ ^^ ***** ------------------------------------------
   E supp su       Event

 that fuller life .
-----------------

 (E=Experiencer, su=supp)
```

Stemmers

UNIT 5

Information Retrieval and Lexical Resources

LECTURE NO:07

APPLICATION OF STEMMING

In information retrieval, text mining SEOs, Web search results, indexing, tagging systems, and word analysis, stemming is employed. For instance, a Google search for prediction and predicted returns comparable results.

TYPES OF STEMMER IN NLTK

There are several kinds of stemming algorithms, and all of them are included in Python NLTK. Let us have a look at them below.

1. PORTER STEMMER – PORTERSTEMMER()

Martin Porter invented the Porter Stemmer or Porter algorithm in 1980. Five steps of word reduction are used in the method, each with its own set of mapping rules. Porter Stemmer is the original stemmer and is renowned for its ease of use and rapidity. Frequently, the resultant stem is a shorter word with the same root meaning.

PorterStemmer() is a module in NLTK that implements the Porter Stemming technique. Let us examine this with the aid of an example.

EXAMPLE OF PORTERSTEMMER()

In the example below, we construct an instance of PorterStemmer() and use the Porter algorithm to stem the list of words.

2. SNOWBALL STEMMER – SNOWBALLSTEMMER()

Martin Porter also created Snowball Stemmer. The method utilized in this instance is more precise and is referred to as “English Stemmer” or “Porter2 Stemmer.” It is somewhat faster and more logical than the original Porter Stemmer.

SnowballStemmer() is a module in NLTK that implements the Snowball stemming technique. Let us examine this form of stemming using an example.

EXAMPLE OF SNOWBALLSTEMMER()

In the example below, we first construct an instance of SnowballStemmer() to use the Snowball algorithm to stem the list of words.

```
from nltk.stem import SnowballStemmer
snowball = SnowballStemmer(language='english')
words = ['generous','generate','generously','generation']
for word in words:
    print(word,"--->",snowball.stem(word))
```

[Out] :

```
generous ---> generous
generate ---> generat
generously ---> generous
generation ---> generat
```

3. LANCASTER STEMMER – LANCASTERSTEMMER()

Lancaster Stemmer is straightforward, although it often produces results with excessive stemming. Over-stemming renders stems non-linguistic or meaningless.

LancasterStemmer() is a module in NLTK that implements the Lancaster stemming technique. Allow me to illustrate this with an example.

Example of LancasterStemmer()

In the example below, we construct an instance of LancasterStemmer() and then use the Lancaster algorithm to stem the list of words.

```
from nltk.stem import LancasterStemmer
lancaster = LancasterStemmer()
words = ['eating','eats','eaten','puts','putting']
for word in words:
    print(word,"--->",lancaster.stem(word))
```

[Out] :

```
eating ---> eat
eats ---> eat
eaten ---> eat
puts ---> put
putting ---> put
```

4. REGEXP STEMMER – REGEXPSTEMMER()

Regex stemmer identifies morphological affixes using regular expressions. Substrings matching the regular expressions will be discarded.

RegexpStemmer() is a module in NLTK that implements the Regex stemming technique. Let us try to understand this with an example.

Example of RegexpStemmer()

In this example, we first construct an object of RegexpStemmer() and then use the Regex stemming method to stem the list of words.

```
from nltk.stem import RegexpStemmer
regexp = RegexpStemmer('ing$|s$|e$|able$', min=4)
words = ['mass','was','bee','computer','advisable']
for word in words:
    print(word,"--->",regexp.stem(word))
```

[Out] :

```
mass ---> mas
was ---> was
```

```
bee ---> bee
computer ---> computer
advisable ---> advis
```

PORTER VS SNOWBALL VS LANCASTER VS REGEX STEMMING IN NLTK

Let us compare the outcomes of several forms of stemming in NLTK using the following

Example :—

```
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer, RegexpStemmer
porter = PorterStemmer()
lancaster = LancasterStemmer()
snowball = SnowballStemmer(language='english')
regexp = RegexpStemmer('ing$|s$|e$|able$', min=4)
word_list = ["friend", "friendship", "friends", "friendships"]
print("{0:20}{1:20}{2:20}{3:30}{4:40}".format("Word","Porter Stemmer","Snowball Stemmer","Lancaster
Stemmer",'Regexp Stemmer'))
for word in word_list:

print("{0:20}{1:20}{2:20}{3:30}{4:40}".format(word,porter.stem(word),snowball.stem(word),lancaster.stem(
word),regexp.stem(word)))
```

[Out] :

```
Word                 Porter Stemmer       Snowball Stemmer     Lancaster Stemmer           Regexp Stemmer
friend               friend               friend               friend                      friend
friendship           friendship           friendship           friend                      friendship
friends              friend               friend               friend                      friend
friendships          friendship           friendship           friend                      friendship
```

STEMMING A TEXT FILE WITH NLTK

We demonstrated stemming certain words before, but what if you have a text file and want to conduct it on the full file? Allow us to comprehend how to do this.

In the example below, we constructed a function called stemming that uses word_tokenize to tokenize the text and then uses SnowballStemmer to stem down the token to its basic form.

We then add it to a list and finally join and return the list's elements.

```
from nltk.tokenize import word_tokenize
from nltk.stem import SnowballStemmer
def stemming(text):
    snowball = SnowballStemmer(language='english')
    list=[]
    for token in word_tokenize(text):
        list.append(snowball.stem(token))
    return ' '.join(list)
with open('text_file.txt') as f:
    text=f.read()
print(stemming(text))
```

[Out] :

analyt vidhya provid a communiti base knowledg portal for analyt and data scienc profession. the aim of the platform is to becom a complet portal serv all knowledg and career need of data scienc profession

CONCLUSION

In this article, we showed you how to conduct stemming in Python using the NLTK package for your natural language processing project. We looked at the several kinds of stemmers available in NLTK, as well as some examples of each. Then we conducted a comparative analysis of the outcomes provided by Porter vs Snowball vs Lancaster vs Regex variants, as well as the results produced by other methods. Finally, we demonstrated how to conduct it on a text file using the NLTK library.